PROGRAMMING ASSIGNMENT 1

Linear Filters

For all programming assignments, please use images from the repository on Canvas. In cases when images in the repository don't meet your requirements, you are free to use an image of your choice. In doing so, please keep in mind these pictures will be public and accessible to anyone in the SJSU domain.

Part 1 (10 points)

Load an image with interesting textures. Smooth the image in several ways using cv2.smooth() with smoothtype=cv2.GAUSSIAN.

a) Use a symmetric 3 × 3, 5 × 5, 9 × 9, and 11 × 11 smoothing window size and display the results.

b) Are the output results nearly the same by smoothing the image twice with a 5 × 5 Gaussian filter as when you smooth once with two 11 × 11 filters? Why or why not?

Part 2 (30 points)

Load an image, and then blur it with cv2.smooth() using a Gaussian filter.

a) Set param1=param2=9. Try several settings of param3 (e.g., 1, 4, and 6). Display the results.

b) Set param1=param2=0 before setting param3 to 1, 4, and 6. Display the results. Are they different? Why?

c) Use param1=param2=0 again, but this time set param3=1 and param4=9. Smooth the picture and display the results.

d) Repeat previous question c) but with param3=9 and param4=1. Display the results.

e) Now smooth the image once with the settings of Exercise 3c and once with the settings of Exercise 3d.
   Display the results.

f) Compare the results in previous question e) with smoothings that use param3=param4=9 and param3=param4=0 (for example using a 9 × 9 filter).
   Are the results the same?
   Why or why not?

Part 3 (15 points)

Load an image of a scene and convert it to grayscale.

a) Run the morphological Top Hat operation on your image and display the results.

b) Convert the resulting image into an 8-bit mask.

c) Copy a grayscale value into the original image where the Top Hat mask (from Part b) of this Problem) is nonzero.

   Display the results.

Part 4 (15 points)

Load an image with many details.

a) Use resize() to reduce the image by a factor of 2 in each dimension (hence the image will be reduced by a factor of 4).

   Do this three times and display the results.

b) Now take the original image and use cv2.pyrDown() to reduce it three times, and then display the results.

c) How are the two results different?

   Why are the approaches different?

Part 5 (10 points)

Load an image of an interesting or sufficiently “rich” scene.

- Using *cv2.threshold(),* set the threshold to 128.
- Use each setting type in Table 5.1 on the image and display the results.

You should familiarize yourself with thresholding functions because they will prove quite useful.

a) Repeat the exercise but use cv2.adaptiveThreshold() instead.

   Set param1=5.

b) Repeat part a) of this exercise using param1=0 and then param1=-5.

Threshold type	Operation
`cv::THRESH_BINARY`	$DST_I = (SRC_I > thresh)\ ?\ MAXVALUE : 0$
`cv::THRESH_BINARY_INV`	$DST_I = (SRC_I > thresh)\ ?\ 0 : MAXVALUE$
`cv::THRESH_TRUNC`	$DST_I = (SRC_I > thresh)\ ?\ THRESH : SRC_I$
`cv::THRESH_TOZERO`	$DST_I = (SRC_I > thresh)\ ?\ SRC_I : 0$
`cv::THRESH_TOZERO_INV`	$DST_I = (SRC_I > thresh)\ ?\ 0 : SRC_I$

Table 5.1. *thresholdType* options for *cv2.threshold()*

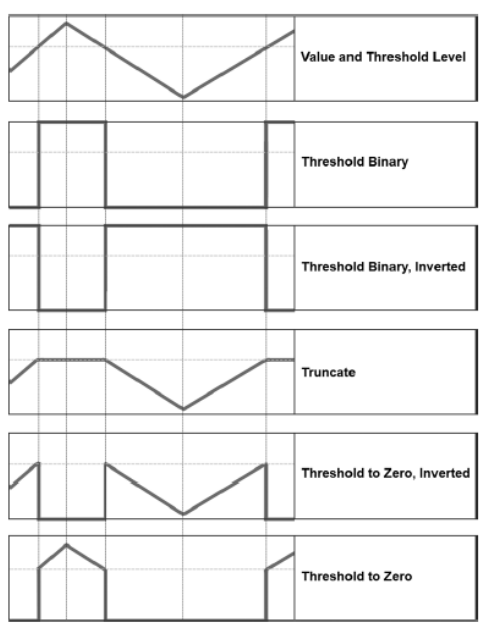

Figure 4.1. Results of varying the threshold type in *cv2.threshold()*

Part 6 (15 points)

Separable kernels: create a 3 × 3 Gaussian kernel using rows [(1/16, 2/16, 1/16), (2/16, 4/16, 2/16), (1/16, 2/16, 1/16)] and with anchor point in the middle.

a) Run this kernel on an image and display the results.

b) Now create two one-dimensional kernels with anchors in the center: one going "across" (1/4, 2/4, 1/4), and one going down (1/4, 2/4, 1/4).

   Load the same original image and use cv2.filter2D() to convolve the image twice, once with the first 1D kernel and once with the second 1D kernel.

   Describe the results.

c) Describe the order of complexity (number of operations) for the kernel in part a and for the kernels in part b) of this exercise.

   The difference is the advantage of being able to use separable kernels and the entire Gaussian class of filter, or any linearly decomposable filter that is separable, since convolution is a linear operation.